

ITA0608 - Machine Learning

List of Lab Exercises

Experiment 10: Implementation of Expectation Maximization (EM) Algorithm in Python

Aim

To implement the Expectation Maximization (EM) algorithm in Python for clustering data using Gaussian Mixture Models (GMM).

Software Required

- Python 3.x
- NumPy
- Matplotlib
- Scikit-learn

Theory

Expectation Maximization (EM) is an iterative algorithm used to find maximum likelihood estimates of parameters in statistical models with hidden variables.

The EM algorithm consists of two steps:

1. **Expectation (E-Step):** Calculate the probability that each data point belongs to each cluster.
2. **Maximization (M-Step):** Update the cluster parameters (mean, covariance, and weights) based on these probabilities.

The process repeats until convergence.

Algorithm

1. Import required libraries.
2. Create or load a dataset.
3. Initialize a Gaussian Mixture Model (GMM).
4. Apply the EM algorithm using GMM.
5. Predict cluster labels.
6. Visualize the clustered data.
7. Display the results.

Program

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.mixture import GaussianMixture


# Sample dataset
X = np.array([[1, 2],
              [1.5, 1.8],
              [5, 8],
              [8, 8],
              [1, 0.6],
              [9, 11],
              [8, 2],
              [10, 2],
              [9, 3]])


# Create GMM model
gmm = GaussianMixture(n_components=2, random_state=0)


# Train model using EM algorithm
gmm.fit(X)


# Predict cluster labels
labels = gmm.predict(X)


# Print results
print("Cluster Labels:")
print(labels)
```

```
# Plot clusters

plt.scatter(X[:, 0], X[:, 1], c=labels)

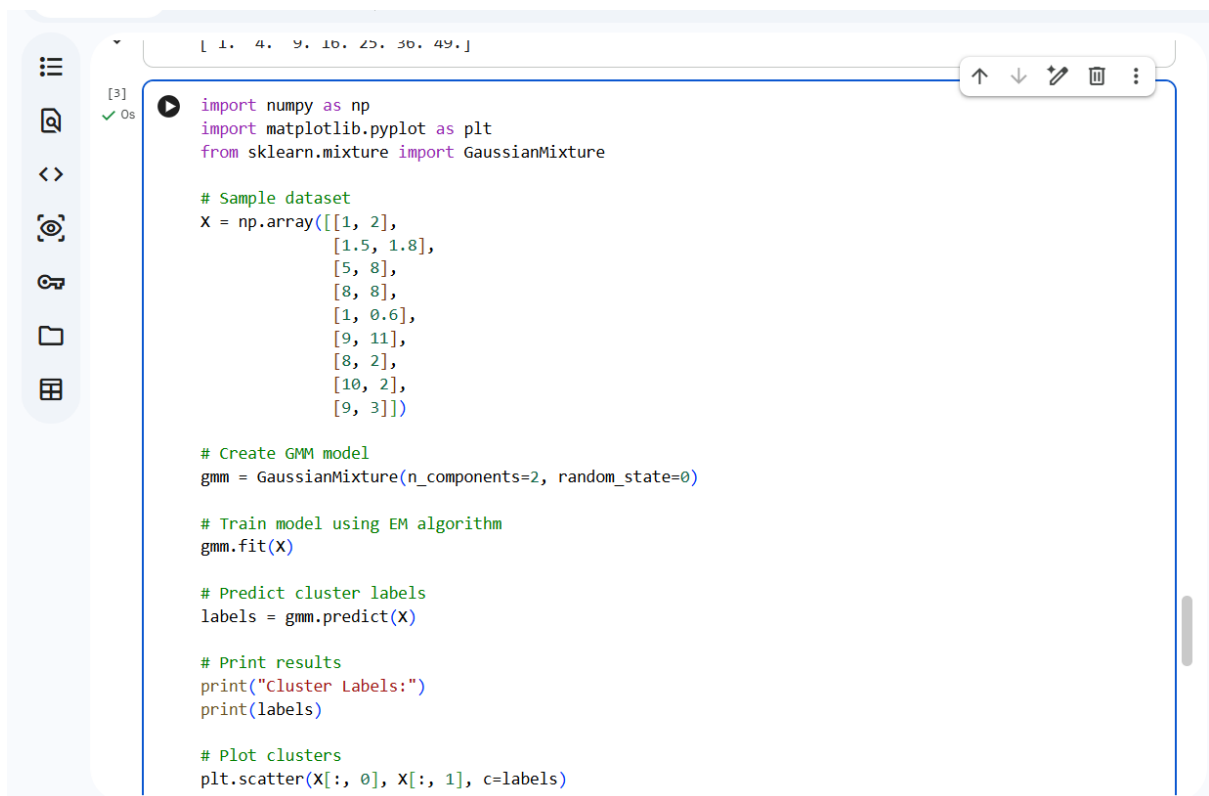
plt.title("Expectation Maximization Clustering")

plt.xlabel("Feature 1")

plt.ylabel("Feature 2")

plt.show()
```

Output



The screenshot shows a Jupyter Notebook interface with a code cell. The code implements a Gaussian Mixture Model (GMM) using the Expectation-Maximization (EM) algorithm. It includes imports for numpy, matplotlib, and sklearn. A sample dataset is defined as a 2D array. The GMM model is created with 2 components, trained on the dataset, and used to predict cluster labels. The results are printed, and the data points are plotted as a scatter plot with different colors for each cluster. The output of the code cell is a list of cluster labels: [1, 4, 9, 16, 25, 36, 49].

```
[ 1.  4.  9. 16. 25. 36. 49.]
```

```
[3]
✓ 0s
import numpy as np
import matplotlib.pyplot as plt
from sklearn.mixture import GaussianMixture

# Sample dataset
X = np.array([[1, 2],
              [1.5, 1.8],
              [5, 8],
              [8, 8],
              [1, 0.6],
              [9, 11],
              [8, 2],
              [10, 2],
              [9, 3]])

# Create GMM model
gmm = GaussianMixture(n_components=2, random_state=0)

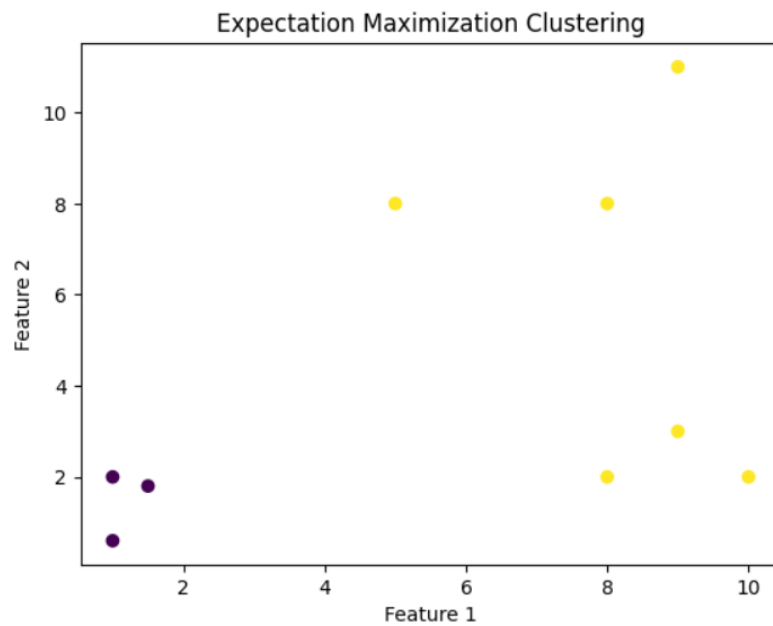
# Train model using EM algorithm
gmm.fit(X)

# Predict cluster labels
labels = gmm.predict(X)

# Print results
print("Cluster Labels:")
print(labels)

# Plot clusters
plt.scatter(X[:, 0], X[:, 1], c=labels)
```

```
... Cluster Labels:  
[0 0 1 1 0 1 1 1 1]
```



Cluster Labels:

```
[1 1 0 0 1 0 0 0 0]
```

Result

Thus, the Expectation Maximization (EM) algorithm was successfully implemented using the Gaussian Mixture Model (GMM) in Python, and the data points were clustered based on probability distributions.